Guía de estudio en español

Basada en L. R. Rabiner, "*A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition*", Proceedings of the IEEE, vol. 77, n.º 2, feb. 1989, pp. 257–286.
Resumen y explicación propios (no es una traducción). Los números de ecuación coinciden con el artículo original para que puedas leer ambos lado a lado.
Proyecto final — Procesos Estocásticos (603504), Unillanos.

0. Cómo usar esta guía

Sección del artículo	Págs. del artículo	Págs. del PDF	Importancia para el proyecto
II. Cadenas de Markov discretas y extensión a HMM	258–261	2–5	★★★ base conceptual
III-A. Problema 1: Forward / Backward	262–263	6–7	★★★ clasificar letras (tablero)
III-B. Problema 2: Viterbi	263–264	7–8	★★★ fase actual + tablero
III-C. Problema 3: Baum-Welch	264–266	8–10	★★★ entrenamiento offline
IV-A. Modelos izquierda-derecha (Bakis)	~266–267	~10–11	★★ topología elegida
V. Problemas de implementación (escalamiento, múltiples secuencias, inicialización, datos insuficientes)	~272–274	~16–18	★★★ imprescindible para que el código funcione
VI en adelante: reconocimiento de voz	~275–286	~19–30	☆ opcional (es el mismo esquema que usaremos con letras)

Las páginas marcadas con ~ son aproximadas; ubícalas por el título de la sección.

1. Sección II — De las cadenas de Markov a los HMM

1.1 Cadena de Markov "visible"

Un sistema está siempre en uno de **N estados** S_1, …, S_N. En cada instante de tiempo t = 1, 2, 3, … salta (o se queda) a otro estado según unas probabilidades fijas. La **propiedad de Markov** dice que el próximo estado solo depende del estado actual, no de toda la historia:

> $P(q_t = S_j \mid q_{t-1} = S_i,\ q_{t-2} = S_k,\ \ldots) = P(q_t = S_j \mid q_{t-1} = S_i) = a_{ij}$

Las a_{ij} forman la **matriz de transición A**: cada $a_{ij} \geq 0$ y cada fila suma 1. El artículo usa el ejemplo del **clima** (lluvioso, nublado, soleado): si sabes el clima de hoy, puedes calcular la probabilidad de una semana completa multiplicando transiciones. Aquí **los estados se ven directamente**: sabes si llueve o no.

Un resultado útil del ejemplo: la probabilidad de quedarse **exactamente d días** en el mismo estado i es $(a_{ii})^{d-1}(1 - a_{ii})$, y la **duración esperada** es

$$\bar{d}_i = 1 / (1 - a_{ii})$$

En el proyecto: esta fórmula es la que usaremos en la **proyección** para saber cuántos pasos dura cada fase de la letra.

1.2 ¿Qué pasa cuando los estados están ocultos?

Ahora no vemos el estado, solo un **símbolo** que el estado "emite". El artículo da dos ejemplos:

- **Lanzamiento de monedas:** detrás de una cortina, alguien lanza una o varias monedas (quizá trucadas) y solo te dice "cara" o "sello". No sabes qué moneda usó en cada lanzamiento.
- **Urnas y bolas:** hay N urnas, cada una con bolas de M colores en proporciones distintas. Alguien elige una urna (según una cadena de Markov), saca una bola, te dice el color, la devuelve y repite. Tú ves solo la secuencia de colores. **Lo oculto es la urna**; **lo observado es el color**.

1.3 Los 5 elementos de un HMM

Símbolo	Qué es	Urnas y bolas	Nuestro proyecto
N	número de estados ocultos	número de urnas	fases del trazo (6–10)
M	número de símbolos observables $V = \{v_1, \ldots, v_M\}$	colores de bola	17 = 16 direcciones + "levantar lápiz"
$A = \{a_{ij}\}$	prob. de pasar del estado i al j	cómo se elige la siguiente urna	quedarse en la fase o avanzar
$B = \{b_j(k)\}$	prob. de emitir v_k estando en j	proporción de colores en la urna j	qué direcciones son típicas en cada fase
$\pi = \{\pi_i\}$	prob. de empezar en i	qué urna se usa primero	(1, 0, …, 0): siempre se empieza en la fase 1

Notación compacta: $\boldsymbol{\lambda = (A, B, \pi)}$. Una secuencia observada se escribe $O = O_1\ O_2 \ldots O_T$ y una secuencia de estados $Q = q_1\ q_2 \ldots q_T$.

Cómo genera datos un HMM (procedimiento del artículo): (1) elegir q_1 según π; (2) emitir O_t según $b_{q_t}(\cdot)$; (3) pasar a q_{t+1} según la fila q_t de A; (4) repetir hasta T.

2. Los tres problemas básicos (sección II-C)

#	Pregunta	Algoritmo	Uso en el proyecto
1	**Evaluación:** dado λ y O, ¿cuánto vale P(O | λ)?	Forward (o Backward)	Calcular qué tan bien explica cada letra el trazo → ranking
2	**Decodificación:** dado λ y O, ¿cuál es la secuencia de estados Q "óptima"?	Viterbi	Saber en qué fase va el usuario → proyección
3	**Aprendizaje:** ¿cómo ajusto λ para maximizar P(O | λ)?	Baum-Welch	Entrenamiento offline de las 27 letras

El esquema de reconocimiento que propone el artículo (para palabras habladas) es exactamente el nuestro con letras: **un HMM por clase**, entrenado con el problema 3; para clasificar, se evalúa $P(O \mid \lambda_c)$ en cada modelo (problema 1) y se elige el mayor.

3. Problema 1 — Evaluación (sección III-A)

3.1 La forma "ingenua" (ecs. 12–17)

Para una secuencia de estados fija Q, y suponiendo que las observaciones son independientes dado el estado:

$$P(O \mid Q, \lambda) = b_{q1}(O_1) \cdot b_{q2}(O_2) \cdots b_{qT}(O_T) \quad \textbf{(13b)}$$

$$P(Q \mid \lambda) = \pi_{q1} \cdot a_{q1q2} \cdot a_{q2q3} \cdots a_{qT-1\,qT} \quad \textbf{(14)}$$

$$P(O \mid \lambda) = \Sigma_{\text{todas las } Q} P(O \mid Q, \lambda) \cdot P(Q \mid \lambda) \quad \textbf{(16–17)}$$

Problema: hay $\mathbf{N^T}$ secuencias posibles. Con N = 5 y T = 100 son unas 10^{72} operaciones. Imposible.

3.2 Algoritmo Forward (ecs. 18–21) ★ tablero

Idea: en vez de recorrer cada camino por separado, ir acumulando "cuánta probabilidad llega" a cada estado en cada instante. Se define la **variable hacia adelante**:

$$\alpha_t(i) = P(O_1\, O_2 \cdots O_t,\ q_t = S_i \mid \lambda) \quad \textbf{(18)}$$

En palabras: probabilidad de haber visto las primeras t observaciones **y** estar ahora en el estado i.

$$\text{1) Inicialización:} \quad \alpha_1(i) = \pi_i \cdot b_i(O_1), \qquad 1 \le i \le N \quad \textbf{(19)}$$

$$\text{2) Inducción:} \quad \alpha_{t+1}(j) = \left[\Sigma_{i=1..N}\, \alpha_t(i) \cdot a_{ij} \right] \cdot b_j(O_{t+1})$$
$$1 \le t \le T-1, \quad 1 \le j \le N \quad \textbf{(20)}$$

$$\text{3) Terminación:} \quad P(O \mid \lambda) = \Sigma_{i=1..N}\, \alpha_T(i) \quad \textbf{(21)}$$

Lectura de la inducción (20): para llegar al estado j en t+1, pude venir de cualquier estado i. Tomo lo que había acumulado en i ($\alpha_t(i)$), lo multiplico por la probabilidad de saltar de i a j (a_{ij}), sumo sobre todos los i, y finalmente multiplico por la probabilidad de que j emita lo que realmente observé ($b_j(O_{t+1})$).

Costo: del orden de $\mathbf{N^2T}$ operaciones (≈ 2500 para N = 5, T = 100) en lugar de 10^{72}. La figura 4 del artículo muestra el **enrejado (trellis)**: columnas = tiempo, filas = estados; todos los caminos "se funden" en los N nodos de cada columna.

> **En el proyecto:** (a) para clasificar mientras el usuario dibuja usamos la terminación (21) con t en lugar de T: $P(O_{1:t} \mid \lambda_c) = \Sigma_i\, \alpha_t(i)$. (b) La inducción (20) es justamente el "Forward incremental": con cada nueva flecha se aplica un solo paso de (20).

3.3 Algoritmo Backward (ecs. 23–25)

$$\beta_t(i) = P(O_{t+1}\, O_{t+2} \cdots O_T \mid q_t = S_i, \lambda) \qquad (23)$$

1) Inicialización: $\beta_T(i) = 1, \quad 1 \le i \le N$

2) Inducción: $\beta_t(i) = \Sigma_{j=1..N}\, a_{ij} \cdot b_j(O_{t+1}) \cdot \beta_{t+1}(j), \quad t = T-1, \ldots, 1$ (24)

(25)

Es el "espejo" de Forward: mira desde t hacia el final. No hace falta para el problema 1, pero **sí para el problema 3** (entrenamiento).

4. Problema 2 — Decodificación (sección III-B)

4.1 Estado más probable en cada instante (ecs. 26–29)

$$\gamma_t(i) = P(q_t = S_i \mid O, \lambda) = \alpha_t(i)\, \beta_t(i) \,/\, \Sigma_j\, \alpha_t(j)\, \beta_t(j) \qquad (26–27)$$

$$q_t = \operatorname{argmax}_i\, \gamma_t(i) \qquad (29)$$

Inconveniente: elegir el mejor estado en cada t por separado puede producir una secuencia **imposible** (por ejemplo, un salto con $a_{ij} = 0$). Por eso se prefiere buscar el **mejor camino completo**: Viterbi.

4.2 Algoritmo de Viterbi (ecs. 30–35) ★ tablero

$$\delta_t(i) = \max_{q1\ldots qt-1} P(q_1 \cdots q_t = i, O_1 \cdots O_t \mid \lambda) \qquad (30)$$

$\delta_t(i)$ es la probabilidad del **mejor camino único** que termina en i en el instante t. $\psi_t(j)$ guarda **de dónde vine** en ese mejor camino.

$$
\begin{aligned}
&\text{1) Inicialización:} && \delta_1(i) = \pi_i\, b_i(O_1), \quad \psi_1(i) = 0 && \\
&\text{2) Recursión:} && \delta_t(j) = \max_i \left[\delta_{t-1}(i)\, a_{ij} \right] \cdot b_j(O_t) && \text{(32a–b)} \\
& && \psi_t(j) = \operatorname{argmax}_i \left[\delta_{t-1}(i)\, a_{ij} \right] && \text{(33a)} \\
&\text{3) Terminación:} && P^* = \max_i \delta_T(i), \quad q_T^* = \operatorname{argmax}_i \delta_T(i) && \text{(33b)} \\
&\text{4) Retroceso:} && q_t^* = \psi_{t+1}(q_{t+1}^*), \quad t = T-1, T-2, \ldots, 1 && \text{(34a–b)} \\
& && && \text{(35)}
\end{aligned}
$$

Clave para el tablero: Viterbi es igual a Forward salvo dos cosas: (1) cambia la **suma Σ** por un **máximo**; (2) guarda ψ para poder reconstruir el camino al final (retroceso). Forward responde "¿qué tan probable es todo lo que vi, sumando todos los caminos?"; Viterbi responde "¿cuál es el único camino más probable?".

El artículo menciona que en la práctica Viterbi se hace con **logaritmos**: los productos se vuelven sumas y no hay problemas numéricos.

5. Problema 3 — Entrenamiento: Baum-Welch (sección III-C)

No existe una fórmula cerrada para el mejor λ. Baum-Welch es un procedimiento **iterativo** (es un caso del algoritmo EM): parte de un λ inicial y lo mejora en cada vuelta. Solo garantiza un **máximo local**.

5.1 Las variables auxiliares (ecs. 36–39)

$$
\begin{aligned}
\xi_t(i,j) &= P(q_t = S_i,\, q_{t+1} = S_j \mid O, \lambda) \\
&= \alpha_t(i) \cdot a_{ij} \cdot b_j(O_{t+1}) \cdot \beta_{t+1}(j) \,/\, P(O \mid \lambda) && \text{(36–37)} \\
\gamma_t(i) &= \Sigma_j\, \xi_t(i,j) && \text{(38)}
\end{aligned}
$$

$\xi_t(i,j)$: probabilidad de estar en i en t **y** en j en t+1 (figura 6). $\gamma_t(i)$: probabilidad de estar en i en t. Sumándolas en el tiempo obtenemos **conteos esperados**:

$$
\begin{aligned}
&\Sigma_{t=1..T-1}\, \gamma_t(i) && = \text{número esperado de transiciones que salen de } S_i \\
&\Sigma_{t=1..T-1}\, \xi_t(i,j) && = \text{número esperado de transiciones de } S_i \text{ a } S_j && \text{(39a)} \\
& && && \text{(39b)}
\end{aligned}
$$

5.2 Fórmulas de reestimación (ecs. 40a–c) ★

$$
\begin{aligned}
\bar{\pi}_i &= \gamma_1(i) && \text{(40a)} \\
\bar{a}_{ij} &= \Sigma_{t=1..T-1}\, \xi_t(i,j) \;/\; \Sigma_{t=1..T-1}\, \gamma_t(i) && \text{(40b)} \\
\bar{b}_j(k) &= \Sigma_{t\,:\,O_t = v_k}\, \gamma_t(j) \;/\; \Sigma_{t=1..T}\, \gamma_t(j) && \text{(40c)}
\end{aligned}
$$

Intuición: son simplemente "proporciones de conteos", como si contaras a mano, pero con conteos **esperados** (porque no sabes con certeza en qué estado estabas):

- $\bar{a}_{ij}$ = (veces que esperamos ir de i a j) / (veces que esperamos salir de i).

- $\bar{b}_j(k)$ = (veces que esperamos estar en j viendo el símbolo k) / (veces que esperamos estar en j).

Garantía (ec. 42): con el nuevo modelo $\bar{\lambda}$ se cumple $P(O \mid \bar{\lambda}) \geq P(O \mid \lambda)$. Se repite hasta que la verosimilitud deje de crecer. Las fórmulas mantienen automáticamente las restricciones (ec. 43): π suma 1, cada fila de A suma 1, cada fila de B suma 1.

> **En el proyecto:** un test excelente para el día 4 es verificar que log $P(O \mid \lambda)$ **nunca baja** entre iteraciones. Si baja, hay un error en el código.

6. Modelos izquierda-derecha o de Bakis (sección IV-A)

El artículo distingue el modelo **ergódico** (cualquier estado puede ir a cualquier otro) del modelo **izquierda-derecha**, donde el índice del estado nunca disminuye. Es el adecuado para señales que **avanzan en el tiempo** (voz… y trazos de letras). Sus propiedades:

```
a_ij = 0   si j < i                  (no se puede retroceder)
π_i = 1 si i = 1,  0 si i ≠ 1        (siempre se empieza en el primer estado)
a_NN = 1,  a_Ni = 0 para i < N       (el último estado es absorbente)
a_ij = 0   si j > i + Δ              (límite de saltos; p. ej. Δ = 2)
```

Dato importante: si un a_{ij} empieza en 0, **Baum-Welch lo mantiene en 0** (porque $\xi_t(i,j) = 0$). Así, la estructura izquierda-derecha se conserva sola durante el entrenamiento.

> **En el proyecto:** usaremos Δ = 1 o 2 (quedarse, avanzar 1 o saltar 1 fase). Así A tiene muy pocos parámetros y se entrena bien con ~30 ejemplos por letra.

7. Problemas de implementación (sección V) — imprescindible para programar

7.1 Escalamiento (V-A) ★ tablero

$\alpha_t(i)$ es un producto de muchas probabilidades menores que 1, así que **tiende a cero exponencialmente**. Con T ≈ 100 ya se sale del rango de los números de punto flotante (*underflow*) y el computador lo convierte en 0. La solución del artículo: en cada t, **normalizar** los α para que sumen 1 y guardar el factor usado.

```
ᾱ_t(j) = [ Σ_i α̂_{t-1}(i) a_ij ] · b_j(O_t)      (paso de Forward normal)
c_t    = 1 / Σ_j ᾱ_t(j)                          (coeficiente de escala)
α̂_t(j) = c_t · ᾱ_t(j)                            (ahora Σ_j α̂_t(j) = 1)

log P(O | λ) = − Σ_{t=1..T} log c_t
```

Los β se escalan con los **mismos** c_t. En las fórmulas de reestimación (40b–c) los factores de escala se cancelan, así que se pueden usar directamente $\hat{\alpha}$ y β. Para Viterbi, la alternativa es trabajar con **logaritmos** (sumas en lugar de productos).

> **En el proyecto:** la clasificación compara $\log P(O \mid \lambda_c) = -\Sigma \log c_t$ entre las 27 letras. Como los $\hat{\alpha}$ suman 1, también nos dan directamente la **distribución del estado actual**, que usamos para la proyección.

7.2 Múltiples secuencias de entrenamiento (V-B)

Un modelo izquierda-derecha no se puede entrenar con un solo ejemplo (cada estado se visita pocas veces). Con K ejemplos $O^{(1)}, \ldots, O^{(K)}$, la reestimación consiste en **sumar los numeradores de todos los ejemplos** y **sumar los denominadores de todos los ejemplos**, y luego dividir. (Con escalamiento, cada ejemplo usa sus propios c_t.)

$$\bar{a}_{ij} = \Sigma_k \Sigma_t \xi_t^{(k)}(i,j) \;/\; \Sigma_k \Sigma_t \gamma_t^{(k)}(i) \qquad (\text{análogo para } \bar{b}_j(m))$$

7.3 Valores iniciales (V-C)

Baum-Welch solo encuentra máximos locales, así que el punto de partida importa. Según el artículo, π y A pueden empezar uniformes o aleatorios (respetando ceros de la topología), pero **B se beneficia mucho de una buena inicialización**, por ejemplo segmentando manualmente o con **k-medias segmentales**: dividir cada secuencia en N tramos, contar los símbolos de cada tramo y usar esas frecuencias como B inicial; luego alinear con Viterbi y recontar.

> **En el proyecto:** dividir cada trazo en N partes iguales y contar direcciones por parte es una inicialización simple, buena y fácil de explicar.

7.4 Datos insuficientes (V-D)

Si un símbolo nunca aparece en un estado durante el entrenamiento, $b_j(k)$ queda en 0, y luego **cualquier** letra nueva con ese símbolo tendría probabilidad 0 (el modelo "se rompe"). El artículo propone imponer un **piso**: $b_j(k) \geq \varepsilon$ (por ejemplo $\varepsilon = 10^{-3}$) y renormalizar cada fila.

> **En el proyecto:** aplicar el piso ε a B después de cada iteración de Baum-Welch. Es clave porque cada persona dibuja distinto.

8. Resumen: del artículo a nuestro código

Concepto en Rabiner	Ecuaciones	Dónde va en nuestro proyecto
$\lambda = (A, B, \pi)$	sección II	`HMMEngine.py` → clase `HMM`; exportado a `models/*.json`
Forward escalado	18–21 + V-A	`HMM.forward()`, `HMM.log_likelihood()`
Paso de inducción	20	`HMM.forward_step()` → clasificación en tiempo real
Backward escalado	23–25 + V-A	`HMM.backward()` (solo entrenamiento)
Viterbi (log)	30–35	`HMM.viterbi()` → fase actual
Baum-Welch multisecuencia	36–40 + V-B, V-C, V-D	`HMM.fit()`, usado por `training/train.py`
Izquierda-derecha	IV-A	inicialización de A y π
Duración esperada $1/(1-a_{ii})$	sección II-A	`HMM.project()` → línea punteada
Clasificador por máxima verosimilitud	sección II-C / VI	$\text{argmax}_c\ P(O \mid \lambda_c)$ + Bayes → lista ordenada

9. Glosario inglés → español

Inglés	Español	Inglés	Español
Hidden Markov Model	Modelo oculto de Markov / Cadena oculta	forward / backward variable	variable hacia adelante / hacia atrás
state	estado	induction / recursion	inducción / recursión
observation symbol	símbolo de observación	termination	terminación
transition probability	probabilidad de transición	backtracking	retroceso (recuperar el camino)
emission / observation probability	probabilidad de emisión	lattice / trellis	enrejado
initial state distribution	distribución inicial	reestimation	reestimación
likelihood	verosimilitud	expected number of …	número esperado de …
left-right (Bakis) model	modelo izquierda-derecha	ergodic model	modelo ergódico (totalmente conectado)
scaling	escalamiento	underflow	desbordamiento por debajo (número → 0)
local maximum	máximo local	training sequence	secuencia de entrenamiento

10. Autoevaluación (respóndela antes del día 2)

1. En el ejemplo de urnas y bolas, ¿qué es lo oculto y qué lo observado? ¿Y en nuestro proyecto?
2. ¿Qué significa cada elemento de $\lambda = (A, B, \pi)$? ¿Por qué cada fila de A y de B debe sumar 1?
3. Enuncia los 3 problemas básicos con tus palabras y di para qué sirve cada uno en el proyecto.
4. Explica con palabras cada parte de la ecuación (20).
5. ¿Por qué la forma ingenua (17) cuesta del orden de $2T \cdot N^T$ y Forward del orden de N^2T?
6. ¿En qué se diferencian Forward y Viterbi? ¿Para qué sirve ψ?
7. ¿Por qué un a_{ij} que empieza en 0 sigue en 0 después de Baum-Welch?
8. ¿Qué es el underflow y cómo lo resuelve el escalamiento? ¿Cómo se recupera $\log P(O \mid \lambda)$?
9. Si $a_{33} = 0.75$, ¿cuántos pasos dura en promedio la fase 3?

Respuesta de la 9: $1 / (1 - 0.75) = 4$ pasos.